

Sri Lanka Institute of Information Technology



IT3012: Intelligent Agents - Lab 2

Y3.S1.WD.AI.01.02

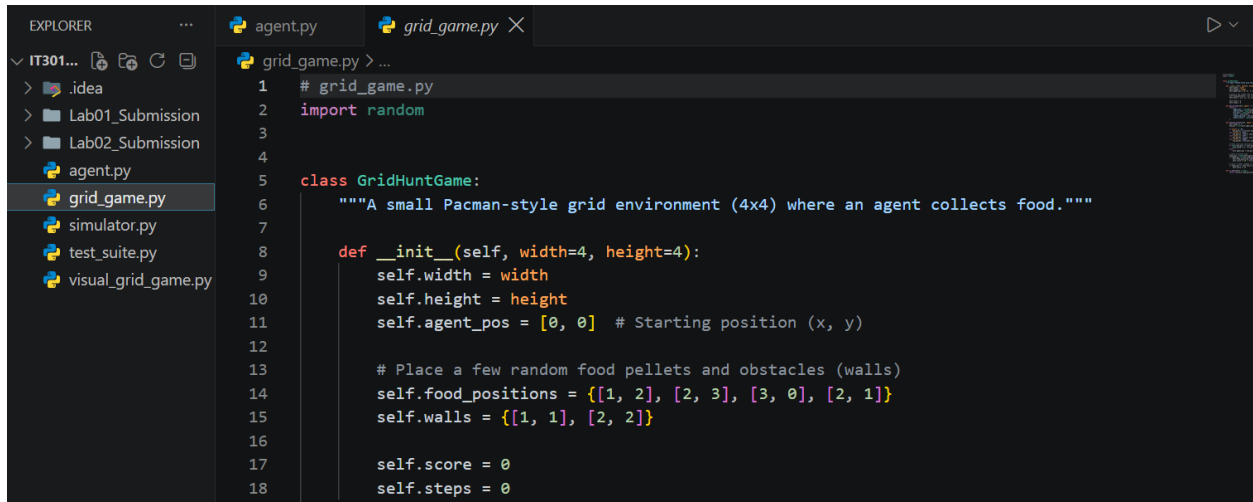
Malabe Campus

Student ID -	IT24103099
Student Name -	Uthpala P.B.H
Module Name -	Intelligent Agents
Module Number -	IT3012
LabSheet Number -	2
LabSheet Date -	2026/August/05

Step 1.1: Setting the Trap (Partial Observability)

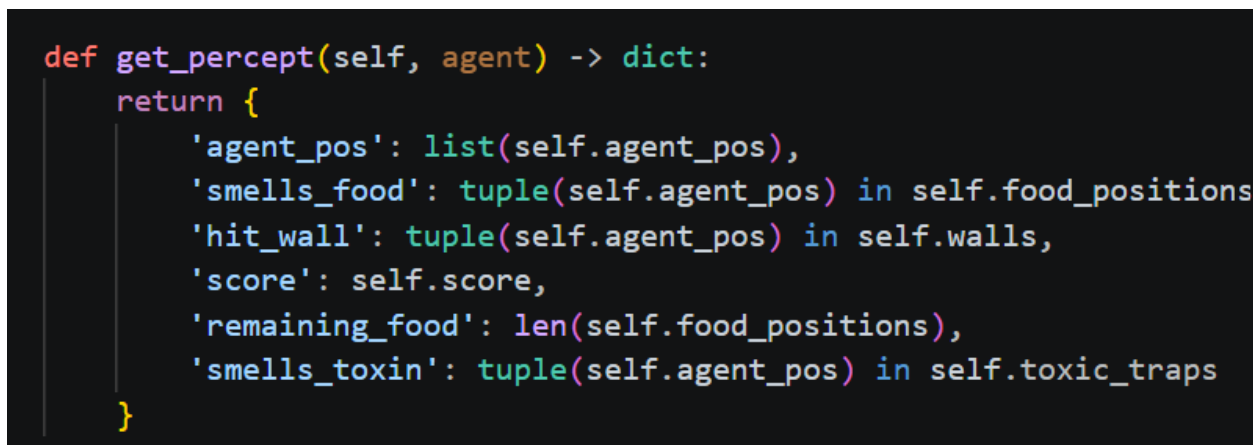
To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.



```
1 # grid_game.py
2 import random
3
4
5 class GridHuntGame:
6     """A small Pacman-style grid environment (4x4) where an agent collects food."""
7
8     def __init__(self, width=4, height=4):
9         self.width = width
10        self.height = height
11        self.agent_pos = [0, 0] # Starting position (x, y)
12
13        # Place a few random food pellets and obstacles (walls)
14        self.food_positions = {[1, 2], [2, 3], [3, 0], [2, 1]}
15        self.walls = {[1, 1], [2, 2]}
16
17        self.score = 0
18        self.steps = 0
```

2. Locate the `get_percept(self)` method.



```
def get_percept(self, agent) -> dict:
    return {
        'agent_pos': list(self.agent_pos),
        'smells_food': tuple(self.agent_pos) in self.food_positions,
        'hit_wall': tuple(self.agent_pos) in self.walls,
        'score': self.score,
        'remaining_food': len(self.food_positions),
        'smells_toxin': tuple(self.agent_pos) in self.toxic_traps
    }
```

3. Modify it so it no longer returns the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

```
agent.py M visual_grid_game.py X
visual_grid_game.py > VisualGridHuntGame > get_percept
6 class VisualGridHuntGame:
54
55     def get_percept(self) -> dict:
56         x, y = self.agent_pos
57
58         # Check the cell directly in front of the agent
59         if self.facing_direction == 'Up':
60             ahead = (x, y + 1)
61         elif self.facing_direction == 'Down':
62             ahead = (x, y - 1)
63         elif self.facing_direction == 'Left':
64             ahead = (x - 1, y)
65         else: # Right
66             ahead = (x + 1, y)
67
68         # Check whether there is a wall ahead
69         wall_ahead = (
70             ahead in self.walls
71             or ahead[0] < 0
72             or ahead[0] >= self.width
73             or ahead[1] < 0
74             or ahead[1] >= self.height
75         )
76
77         # Check whether food is on the current cell
78         food_here = tuple(self.agent_pos) in self.food_positions
79
80         return {
81             'wall_ahead': wall_ahead,
82             'food_here': food_here
83         }
```

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called SimpleReflexAgent .
2. Implement a sense_and_act(self, percept) method that uses strictly IF-THEN logic (Condition-Action rules). Do not use an __init__ method to store history. Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

```
14
15 class SimpleReflexAgent:
16     """Agent that chooses actions using only the current percept."""
17
18     def sense_and_act(self, percept: dict) -> str:
19         # IF food is here THEN stay/move onto it
20         if percept['food_here']:
21             return 'Stay'
22
23         # IF there is a wall ahead THEN turn left
24         if percept['wall_ahead']:
25             return 'Left'
26
27         # ELSE move forward
28         return 'Right'
```

3. Run the simulation. Observe: Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle.

The Simple Reflex Agent can get stuck in a loop because it only reacts to the current percept and has no memory. When it faces the same situation again, it makes the same decision again, causing it to repeat the same actions instead of trying a different path.

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called ModelBasedAgent .

```
class ModelBasedAgent:  
    """Agent that uses current percepts and internal memory."""
```

2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker)

```
def __init__(self):  
    self.visited_cells = set()  
    self.last_action = None
```

3. In `sense_and_act(self, percept)` , first update the state (Transition & Sensor Model) by recording the current percept and the last action taken.

```
def sense_and_act(self, percept: dict) -> str:  
    # Remember the current situation  
    current_state = (  
        percept['wall_ahead'],  
        percept['food_here']  
    )  
    self.visited_cells.add(current_state)
```

4. Update your IF-THEN rules to query the memory. Example Logic: IF wall_ahead AND left_is_visited THEN turn_right

```
# IF food is here THEN collect it
if percept['food_here']:
    self.last_action = 'Stay'
    return 'Stay'

# IF wall ahead and this situation was already visited
# THEN try another direction
if percept['wall_ahead'] and current_state in self.visited_cells:
    self.last_action = 'Right'
    return 'Right'

# IF wall ahead THEN turn left
if percept['wall_ahead']:
    self.last_action = 'Left'
    return 'Left'

# ELSE move forward
self.last_action = 'Right'
return 'Right'
```

5. Run the simulation. Observe: The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

The Model-Based Agent uses its memory to remember previous situations. When it recognizes that it is repeating a situation, it chooses a different action instead of repeating the same movement.

Part 2: Theoretical Evaluation & Lecture Mapping

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

Ans : A Table-Driven Agent would need to store an action for every possible percept sequence. In a complex environment like Chess, the number of possible sequences becomes extremely large due to combinatorial explosion.

As the agent's lifetime increases, its percept history becomes longer, so the number of possible percept sequences grows enormously. Therefore, creating and storing a complete table is practically impossible.

2. (Understand) Look at the code you wrote for your SimpleReflexAgent . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Ans : In our SimpleReflexAgent, these if statements are the Condition-Action Rules:

```
if percept['food_here']:
    return 'Stay'
if percept['wall_ahead']:
    return 'Left'
return 'Right'
```

They mean:

- **IF** food_here → **THEN** Stay
- **IF** wall_ahead → **THEN** Left
- **ELSE** → Right

3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

Ans : The Simple Reflex Agent has **no memory of previous situations**.

Because the environment is **partially observable**, the agent cannot see all the important information about the environment. When it encounters the same percept again, it cannot remember that it has already experienced it.

Same percept → Same rule → Same action → Same situation → Repeat

This can cause an **infinite loop**.

4. (Evaluate) In Step 1.3, you added an internal state to your ModelBasedAgent . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Ans : The Model-Based Agent maintains an internal state/memory using variables such as:

```
self.visited_cells = set()
```

```
self.last_action = None
```

Sensor Model: uses the current percept to understand what the agent currently senses.

Transition Model: uses the remembered state and previous action to help track how the situation changes.

